A PRACTICAL FIELD GUIDE / DATABASE DESIGN

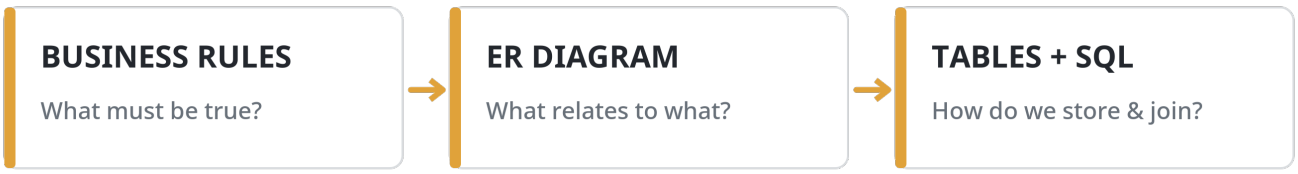
Entity–Relationship Diagrams

From business rules to tables you can confidently JOIN

Original author: Ziyuan (Ryan) Li • Revised November 7, 2026 for BA777 • Instructor: Mohammad Soltaniehha

An ERD is a blueprint for the meaning and structure of data. It shows which kinds of things matter, what we know about them, and how they can be associated.

The design-to-query workflow



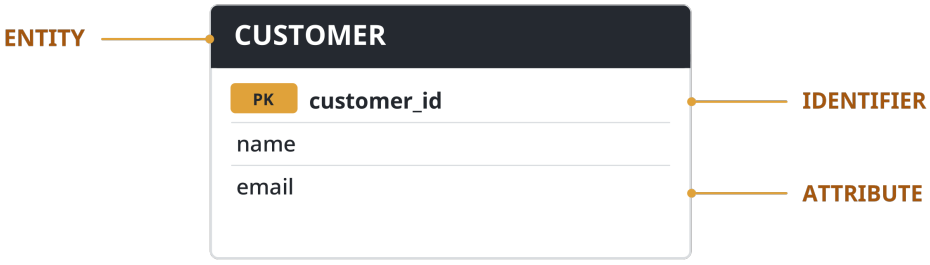
Business rules define the model; the model guides tables, keys, and JOIN conditions.

The core idea A table is not just a collection of columns. Its rows represent a particular kind of entity, and its relationships express business rules.

One ERD, two questions

- Which entities and attributes need to be represented?
- For each relationship, how many related records are allowed—or required?

See the building blocks



CUSTOMER is an entity type; one row is an entity instance.

An entity type has a name, a key, and attributes.

01 The building blocks

Think in terms of business concepts first; then decide how to represent them as tables.

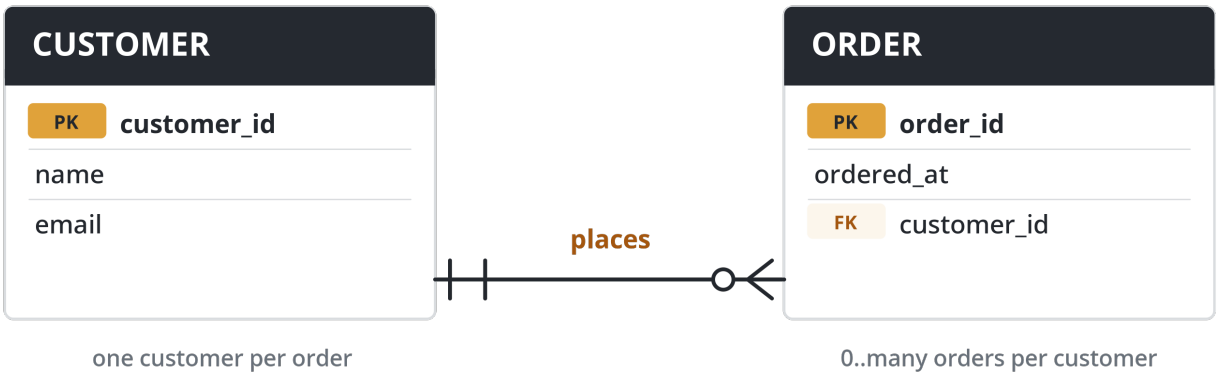
An entity type represents a category of things (for example, CUSTOMER). An entity instance is one particular member of that category (for example, customer 1042). An attribute describes an entity instance.

TERM	PLAIN-LANGUAGE MEANING	EXAMPLE
Entity type	The class of thing we keep data about.	CUSTOMER
Entity instance	One occurrence of an entity type.	Customer 1042
Attribute	A property recorded about an instance.	email
Primary key (PK)	One column or a combination that uniquely identifies each row; no NULLs.	customer_id
Foreign key (FK)	A column or combination referencing a key in a related table (possibly the same table).	ORDER.customer_id
Relationship	An association between entity types, governed by business rules.	CUSTOMER places ORDER

Keys make relationships usable

ORDER.customer_id is a foreign key that references CUSTOMER.customer_id. It lets SQL join an order to its customer without duplicating customer details across orders.

A relationship with two endpoint rules



One CUSTOMER may place 0..N ORDERs; each ORDER must belong to exactly 1 CUSTOMER.

Important distinction A primary key can consist of multiple attributes (a composite key). A foreign key is not automatically a primary key, and a table can contain several foreign keys.

02 Cardinality and participation

A relationship has rules at both ends. Do not infer them from the entity names alone.

For a binary relationship, specify the minimum and maximum number of instances at each end that may be associated with a single instance at the other end. The maximum expresses one versus many; the minimum expresses optional (zero) versus mandatory (one) participation. In textbooks, mandatory and optional are also called total and partial participation, respectively.

Read these four endpoint symbols



Zero or one

optional, at most one

0..1



Exactly one

mandatory, only one

1..1



Zero or many

optional, no upper limit

0..N



One or many

mandatory, no upper limit

1..N

In Crow's Foot notation: circle = zero, bar = one, three-pronged foot = many. Symbols closest to an entity describe that endpoint.

Patterns are assumptions about the business

TTERN	EXAMPLE RULE	READ BOTH ENDS
e-to-one	A person may have 0 or 1 current passport in a particular system; each passport is assigned to 1 person.	0..1 passports per person; 1 person per passport
e-to-many	A customer may place 0 or many orders; each order must belong to 1 customer.	0..N orders per customer; 1 customer pe order
ny-to-many	A student may take many courses; each course may have many students. Both may have zero enrollments.	0..N courses per student; 0..N students per course

Always ask both directions "How many orders can one customer have?" AND "How many customers can one order have?" One-to-many alone does not tell you whether participation is optional.

03 Two common ERD notations

Chen notation emphasizes conceptual relationships; Crow's Foot makes minimum and maximum counts visible on the connecting line.

Chen notation — entities, attributes, relationships



1:N specifies maximum counts; optionality needs additional annotation.

Simplified Chen notation: rectangle = entity, oval = attribute, underlined attribute = key, diamond = relationship; 1 and N show maximum counts.

Crow's Foot notation — relationship endpoints



The same association in Crow's Foot notation, now with explicit minimum as well as maximum cardinality.

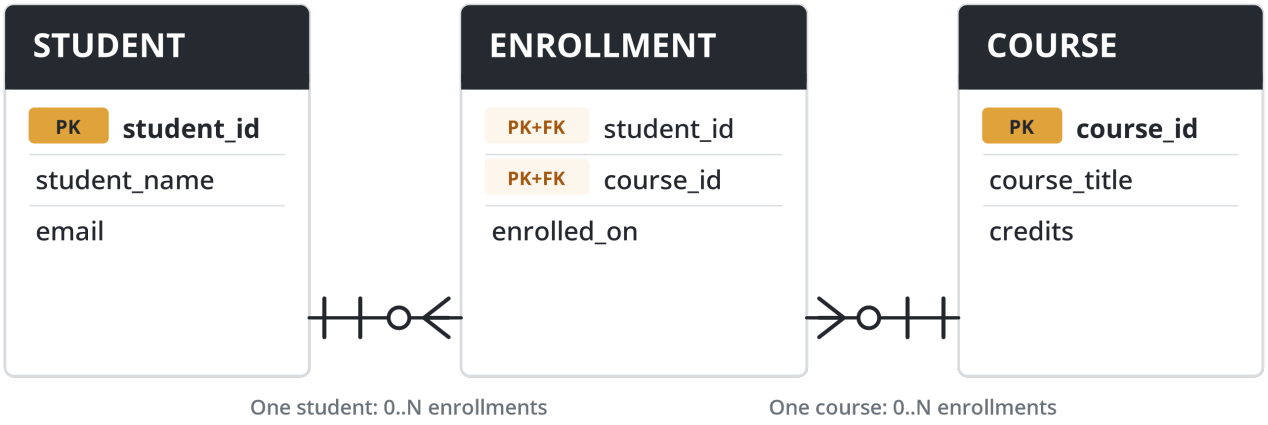
FEATURE	CHEN	CROW'S FOOT
Entity	Rectangle	Rectangle, often listing attributes
Relationship	Named diamond	Line, optionally labeled with verb
Key	Traditionally underlined attribute	Usually labeled PK within the entity
Cardinality	Labels or (min,max) annotations	Symbols at both line ends

Historical context. Peter Chen introduced the entity–relationship model in a foundational 1976 paper [1]. Crow's Foot notation later became widespread in Information Engineering and practical relational database design; notation conventions vary slightly by tool [2].

04 Worked example: students and courses

A many-to-many association is modeled with an enrollment (associative) entity when translated into relational tables.

Business rules: one student may enroll in zero or many courses; one course may have zero or many students. Each enrollment record refers to exactly one student and exactly one course.



STUDENT and COURSE are linked through ENROLLMENT. Each link is one-to-many, replacing the conceptual many-to-many relationship.

What the keys do

TABLE	PRIMARY KEY	FOREIGN KEYS
STUDENT	student_id	—
COURSE	course_id	—
ENROLLMENT	(student_id, course_id)	student_id → STUDENT; course_id → COURSE

The ENROLLMENT table also stores attributes of the relationship, such as enrolled_on or grade. A composite primary key of (student_id, course_id) assumes at most one enrollment per pair; to allow retakes, include a term/attempt identifier or use a separate enrollment_id.

The ERD becomes a JOIN path

```
SELECT s.student_name, c.course_title, e.enrolled_on
FROM ENROLLMENT AS e
JOIN STUDENT AS s ON e.student_id = s.student_id
JOIN COURSE  AS c ON e.course_id  = c.course_id;
```

BigQuery-specific BigQuery can declare primary-key and foreign-key constraints, but does not enforce them. Data quality checks must verify uniqueness and referential integrity [5].

05 How to build a useful ERD

Start with requirements. Draw the diagram only after you can state the business rules clearly.

01	**Set the scope** Name the business process and the questions the database must answer. Example: “Track which students enroll in which courses.”
02	**Identify entities** Find stable business objects and events: STUDENT, COURSE, ENROLLMENT. Prefer singular, descriptive names.
03	**Choose identifiers** Specify a PK for each entity. Record essential attributes, and distinguish identifiers from descriptive fields.
04	**Write relationship rules** Express each rule in both directions and decide minimum and maximum counts (0..1, 1..1, 0..N, or 1..N).
05	**Resolve many-to-many links** Introduce an associative entity where needed; put attributes of the relationship on that entity.
06	**Test realistic scenarios** Use zero-case, one-case, and many-case examples. Check whether the design permits every valid case and rejects invalid ones.

Three mistakes to catch early

COMMON ERROR	BETTER MODELING CHOICE
Treating 1:N as automatically mandatory	Record the minimum separately. A customer can exist without any orders.
Putting a list of course IDs inside STUDENT	Use an ENROLLMENT table; each row represents one association.
Treating an ERD as enforced truth	Validate keys, business rules, and actual source data—especially in analytics platforms.

06 Quick reference and self-check

Use these prompts to read unfamiliar ERDs and to review your own designs.

A five-question review

01 What does one row represent? State the entity instance unambiguously.

02 What uniquely identifies it? Find its primary key, including any composite key.

03 How does it connect to other entities? Find foreign keys and relationship lines.

04 What do both endpoints mean? Read the minimum and maximum counts in both directions.

05 Can the design handle edge cases? Test zero related rows, multiple matches, and repeat transactions.

Check your understanding

QUESTION	ANSWER
A customer has no orders. Is that valid in our example?	Yes. ORDER has a minimum of zero at its end.
Where is customer_id stored as an FK?	In ORDER (the many side of the relationship).
A student can register for several courses; each course has many students. What is needed?	An associative ENROLLMENT entity.
Does a label "1:N" fully specify participation?	No. It gives maximum cardinality, not necessarily the minima.

References and further reading

[1] Chen, P. P.-S. (1976). "The Entity–Relationship Model—Toward a Unified View of Data." ACM Transactions on Database Systems, 1(1), 9–36. doi:10.1145/320434.320440

[2] Microsoft Support. "Create a diagram with Crow's Foot database notation." Notation guide

[3] Painter-Wakefield, C. (2023). A Practical Introduction to Databases, §§2.2–2.4. Colorado School of Mines / Runestone Academy. Open textbook

[4] Li, Q., & Chen, Y.-L. (2009). Modeling and Analysis of Enterprise and Information Systems: From Requirements to Realization, Chapter 6. Springer. Book information

[5] Google Cloud. "Use primary and foreign keys" (BigQuery documentation). Official documentation

A final reminder ERDs are statements about the business, not decorations. When the rules are correct, implementation and SQL become easier to reason about.